

CSE332: Computer Organization & Architecture

Instructor: Mohammad Abdul Qayum, Ph.D.
Department of Electrical Engineering &
Computer Engineering
North South University

Instructor Profile

Previous Experience:

Assistant Professor, Minnesota State University, USA

Investor & Developer, HIVE blockchain based DAO

Postdoctoral fellow, New Mexico State University, USA

PhD (Computer Engineering), New Mexico State University, USA

MSc (Computer Engineering), Oklahoma State University, USA

Rollout Engineer, Banglalink

BSc (Electrical & Electronic Engineering), BUET

Area of Research: Computer Architecture, Embedded Systems, High Performance Computing, Blockchains

CSE332: Computer Organization & Architecture

Text books:

- Computer Organization and Design -The Hardware/Software Interface, 5th Edition (2014)
by David A. Patterson and John L. Hennessy
Morgan Kaufmann Publishers (Elsevier)

MIPS Assembly Language Programming 2003
by Robert Britton, Pearson Publishers



- Course Outline: Uploaded to Canvas

Chapter 1

Computer Abstractions and Technology

Computer History & Basics

- Famous Laws
- Progression of Computing
- Future of Computing
- Computer Components



Moore's Law

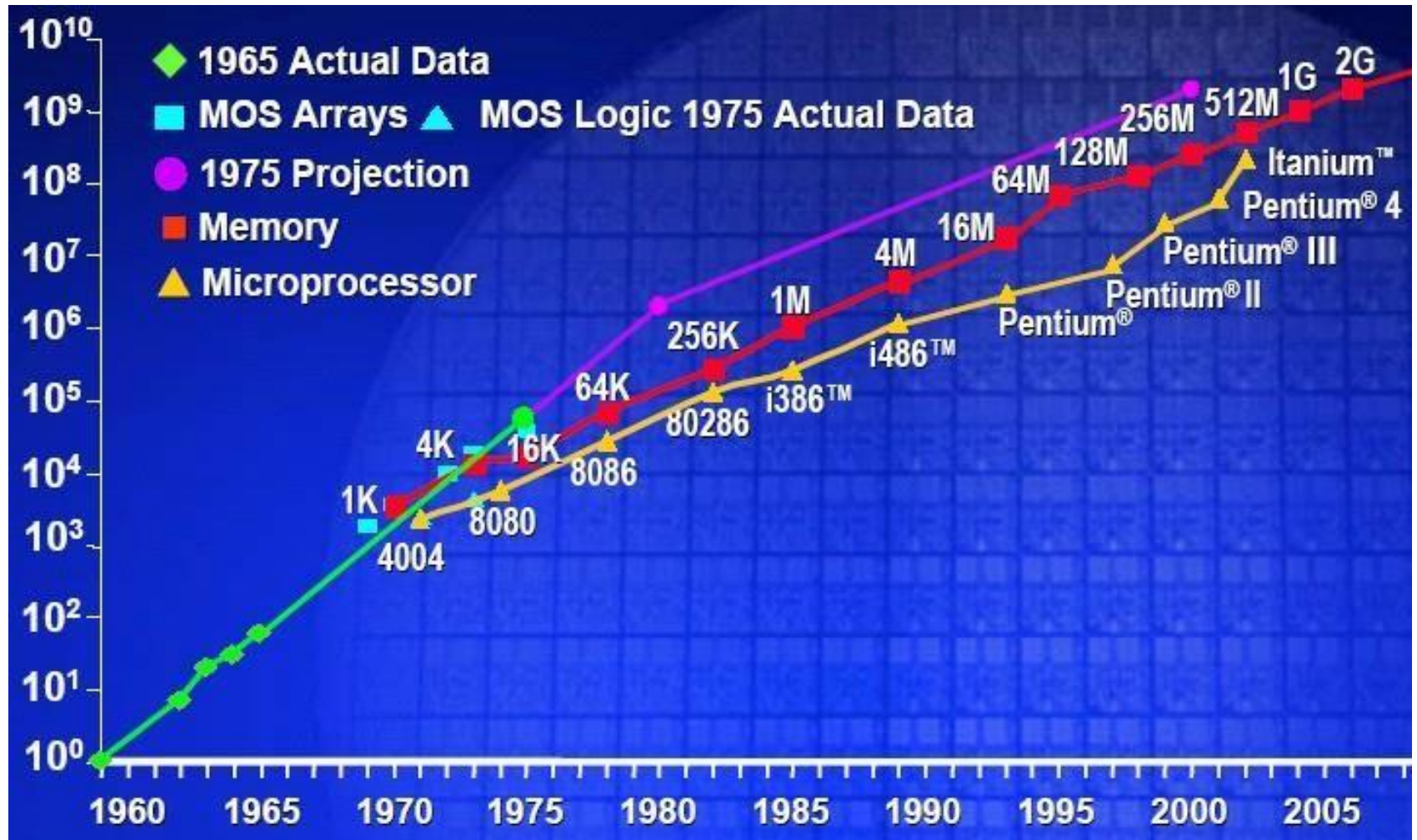
- Moore's Law states that integrated circuit resources double every 18–24 months.
- Moore's Law resulted from a 1965 prediction of such growth in IC capacity made by Gordon Moore, one of the founders of Intel.
- Moore's Law graph to represent designing for rapid change



Moore's Law

	Year of introduction	Transistors
4004	1971	2,250
8008	1972	2,500
8080	1974	5,000
8086	1978	29,000
286	1982	120,000
386™	1985	275,000
486™ DX	1989	1,180,000
Pentium®	1993	3,100,000
Pentium II	1997	7,500,000
Pentium III	1999	24,000,000
Pentium 4	2000	42,000,000

Moore's Law

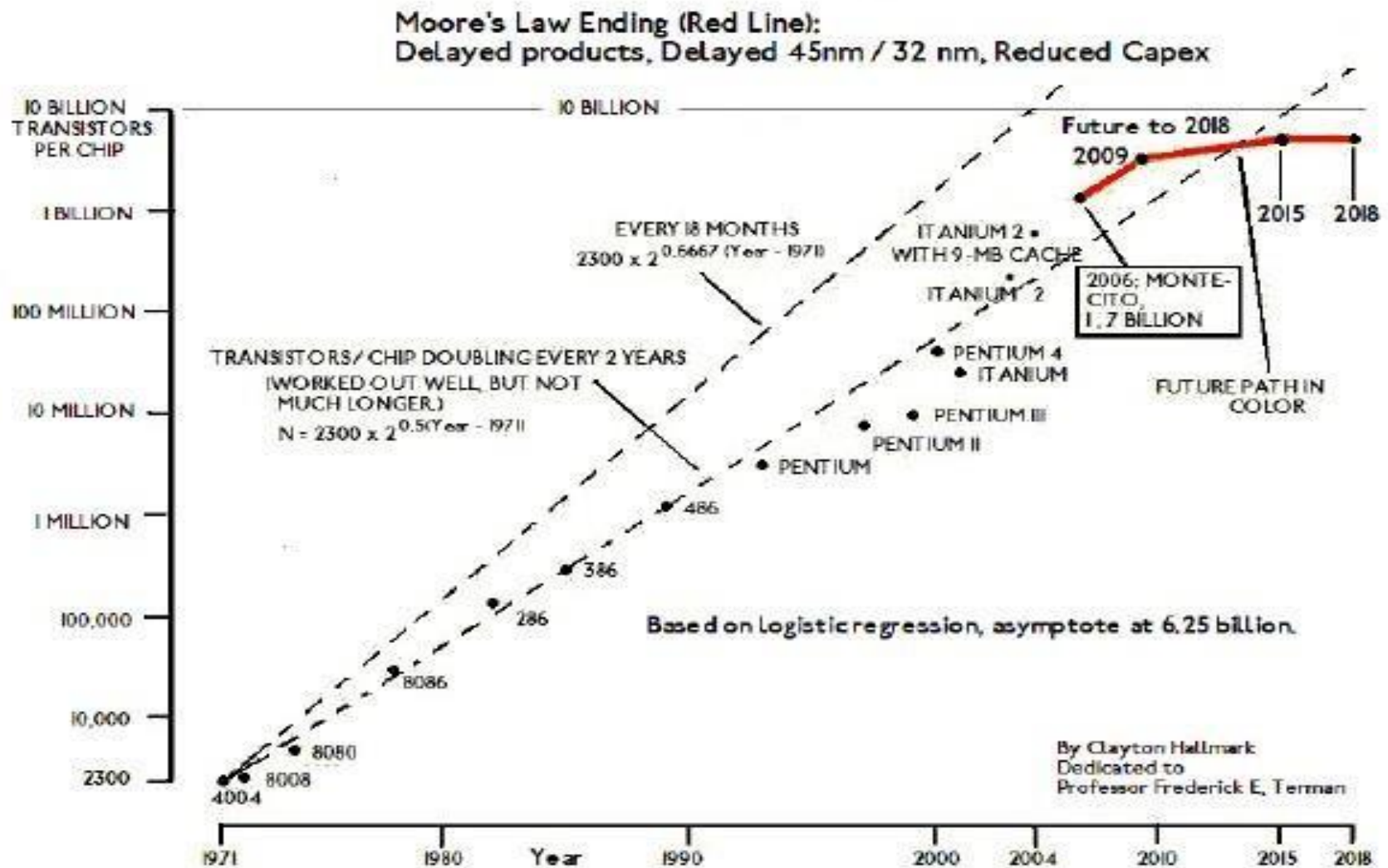


Source: ISSCC 2003 G. Moore "No exponential is forever, but 'forever' can be delayed"

Is Moore's Law Ending?

- ❑ Intel's former chief architect Bob Colwell says: Moore's law will be dead within a decade (August 2013).
- ❑ The end of Moore's Law is on the horizon, says AMD.
- ❑ Theoretical physicist Michio Kaku believes Moore's Law has about 10 years of life left before ever-shrinking transistor sizes smack up against limitations imposed by the laws of thermodynamics and quantum physics (April 2013).

Is Moore's Law Ending? (2)

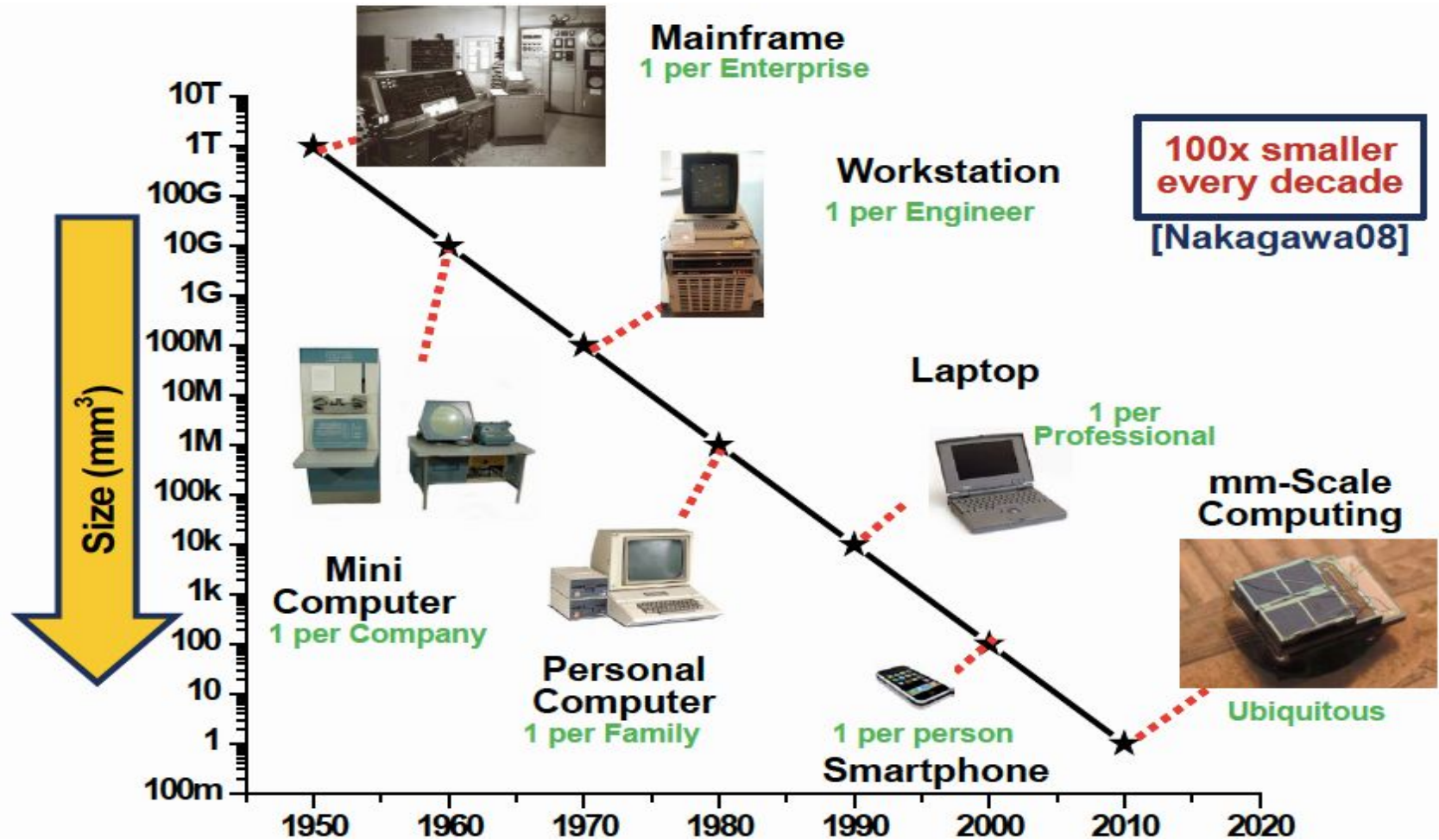


Bell's Law

Bell's Law for the birth and death of computer classes:

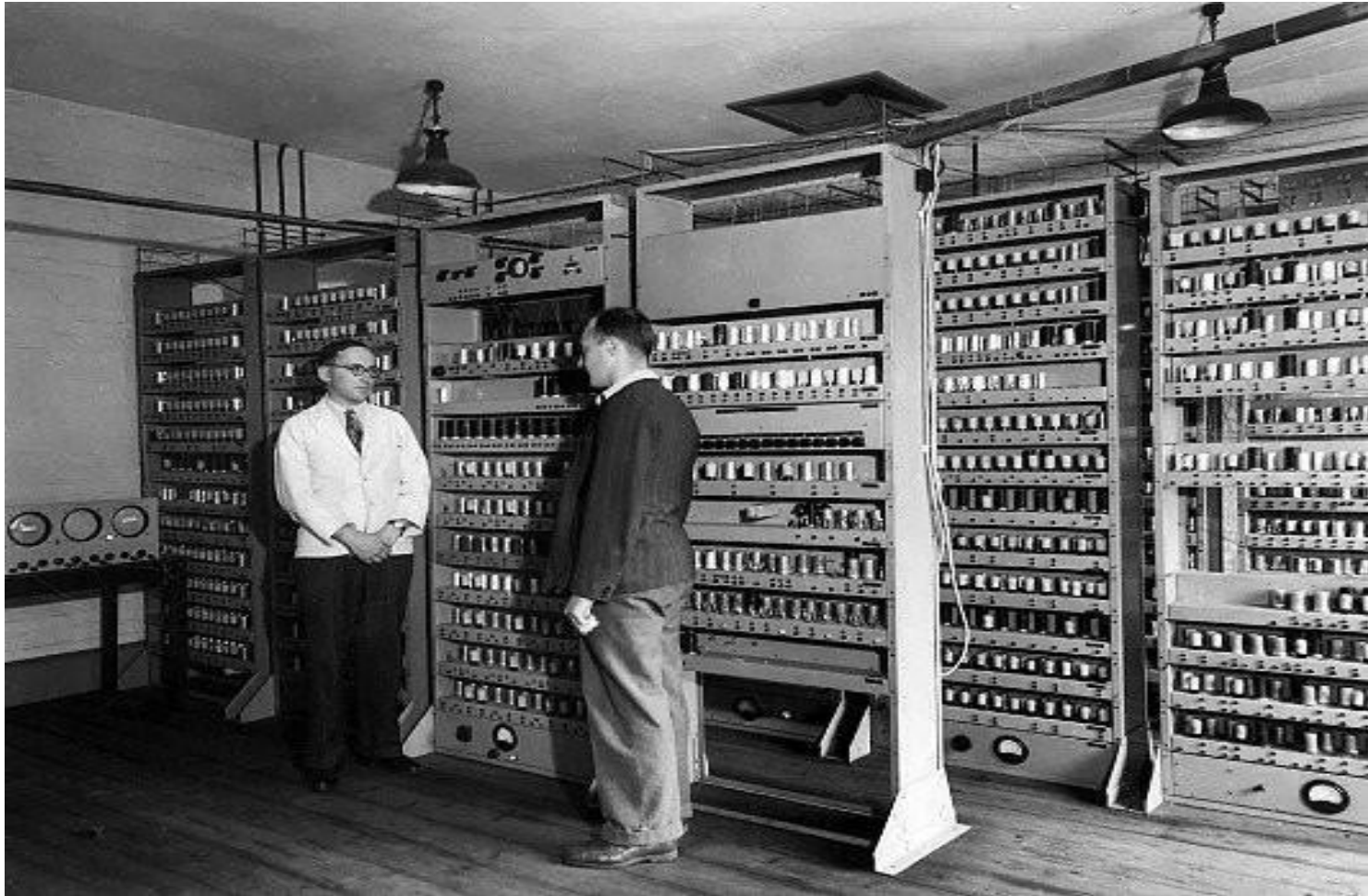
- Bell's law of computer classes formulated by Gordon Bell in 1972 describes how computing systems (computer classes) form, evolve and may eventually die out.
- Roughly every decade a new, lower priced computer class forms based on a new programming platform, network, and interface resulting in new industry.
- In 1951, men could walk inside a computer and now, computers are beginning to “walk” inside of us .

Bell's Law



Source: B Bell, "Bell's Law for the Birth and Death of Computer Classes", Comms of ACM, 2008

The 1st Generation Computer

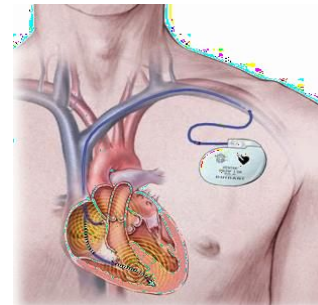


Source:

<http://www.computerhistory.org>

Chapter 1 — Computer Abstractions and Technology — 13

Computers Today



Data Center

Over three years, the power bill for a single server can be higher than the cost of the computer itself.

*Jeffrey W. Clarke
Vice Chairman of Operations & Technology
Sun Microsystems (now Oracle)*

One Google search consumes 0.3 watt-hours.

*Powering a Google search
The Official Google Blog*

The Computer Revolution

- Progress in computer technology
 - Underpinned by Moore's Law
- Makes novel applications feasible
 - Computers in automobiles
 - Cell phones
 - Human genome project
 - World Wide Web
 - Search Engines
- Computers are pervasive (IoT, smartwatch)

The Computer Revolution (2)

- Computers in automobiles (100 uC)
 - reduce pollution, improve fuel efficiency via engine controls, and increase safety through blind spot warnings, lane departure warnings, moving object detection, and airbag inflation to protect occupants in a crash
- Cell phones
 - more than half of the planet having mobile phones, allowing person-to-person communication to almost anyone anywhere in the world

The Computer Revolution (3)

- Human genome project
 - a global effort to identify the estimated 30,000 genes in human DNA to figure out the sequences of the chemical bases that make up human DNA to address ethical, legal, and social issues
 - The cost of computer equipment to map and analyze human DNA sequences was hundreds of millions of dollars. Since, costs continue to drop, we will soon be able to acquire our own genome, allowing medical care to be tailored to us

The Computer Revolution (4)

- World Wide Web
 - has transformed our society. Web has replaced libraries and newspapers
- Search Engines
 - As the content of the web grew in size and in value, many people rely on search engines for such a large part of their lives that it would be a hardship to go without them

Classes of Computers

- Personal computers
- Server computers
- Supercomputers
- Embedded computers

Classes of Computers (2)

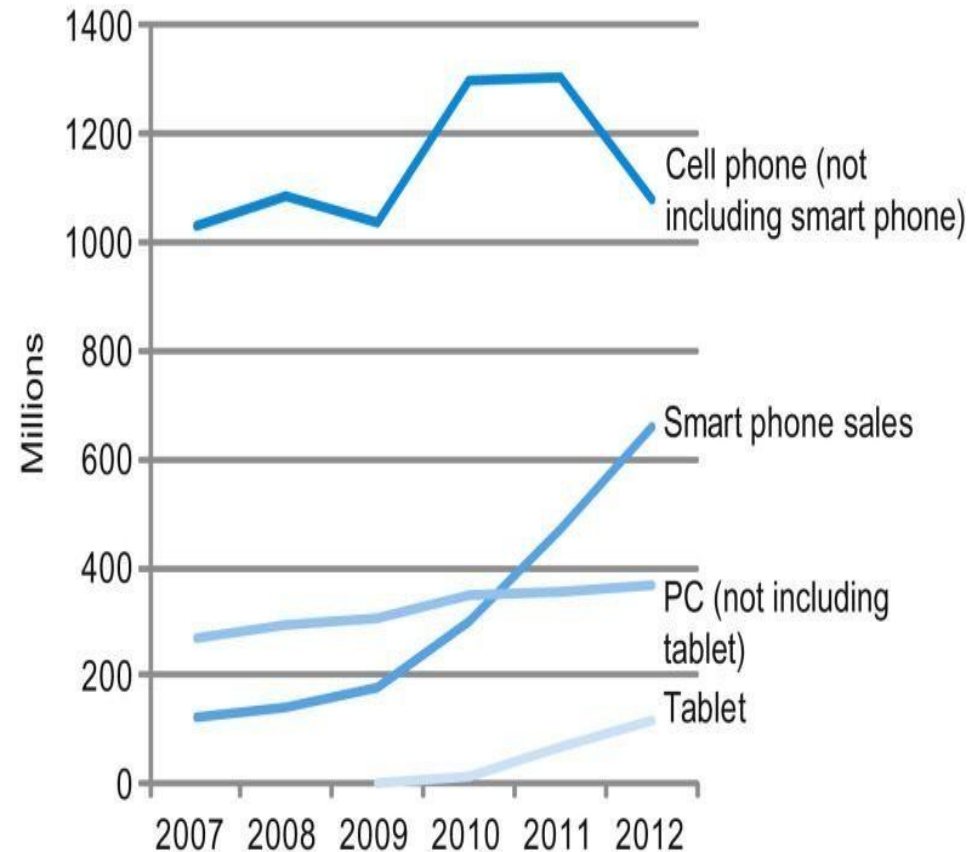
- Personal computers
 - Computers designed for use by an individual
 - General purpose, variety of software
 - Subject to cost/performance tradeoff
- Server computers
 - Computers used for running larger programs for multiple users, often simultaneously
 - Network based
 - High capacity, performance, reliability
 - Range from small servers to building sized

Classes of Computers (3)

- Supercomputers
 - Consist thousands of processors and many terabytes of memory
 - High-level scientific and engineering calculations
 - Highest capability and cost but represent a small fraction of the overall computer market
- Embedded computers
 - Hidden as components of systems
 - Largest class of computers and span the widest range of applications
 - Strict power/performance/cost limitations

The PostPC Era

- ❑ Smart phones represent the recent growth in cell phone industry, and they passed PCs in 2011.
- ❑ Tablets are the fastest growing category, nearly doubling between 2011 and 2012.
- ❑ Recent PCs and traditional cell phone categories are relatively flat or declining.



The PostPC Era (2)

- Personal Mobile Devices (PMDs)
 - Small wireless devices
 - Battery operated
 - Connects to the Internet
 - Hundreds of dollars
 - Smart phones, tablets, electronic glasses



The PostPC Era (3)

- Cloud computing
 - Term cloud essentially used for the Internet
 - Portion of software run on a PMD and a portion run in the Cloud
 - Warehouse Scale Computers (WSC)
 - Big datacenters containing 100,000 servers
 - Amazon and Google cloud vendors
 - Software as a Service (SaaS)
 - Delivers software and data as a service over the Internet
 - Web search and social networking

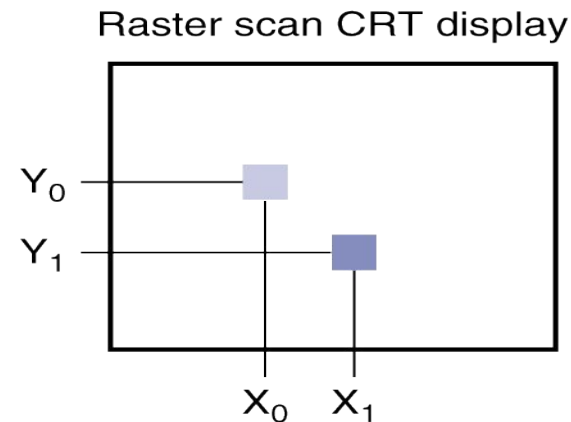
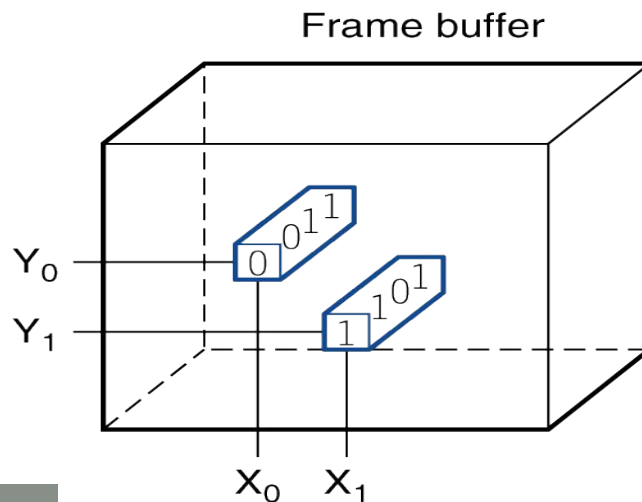
Touchscreen

- PostPC device
- Supersedes keyboard and mouse
- Resistive and Capacitive types
 - Most tablets, smart phones use capacitive
 - Capacitive allows multiple touches simultaneously

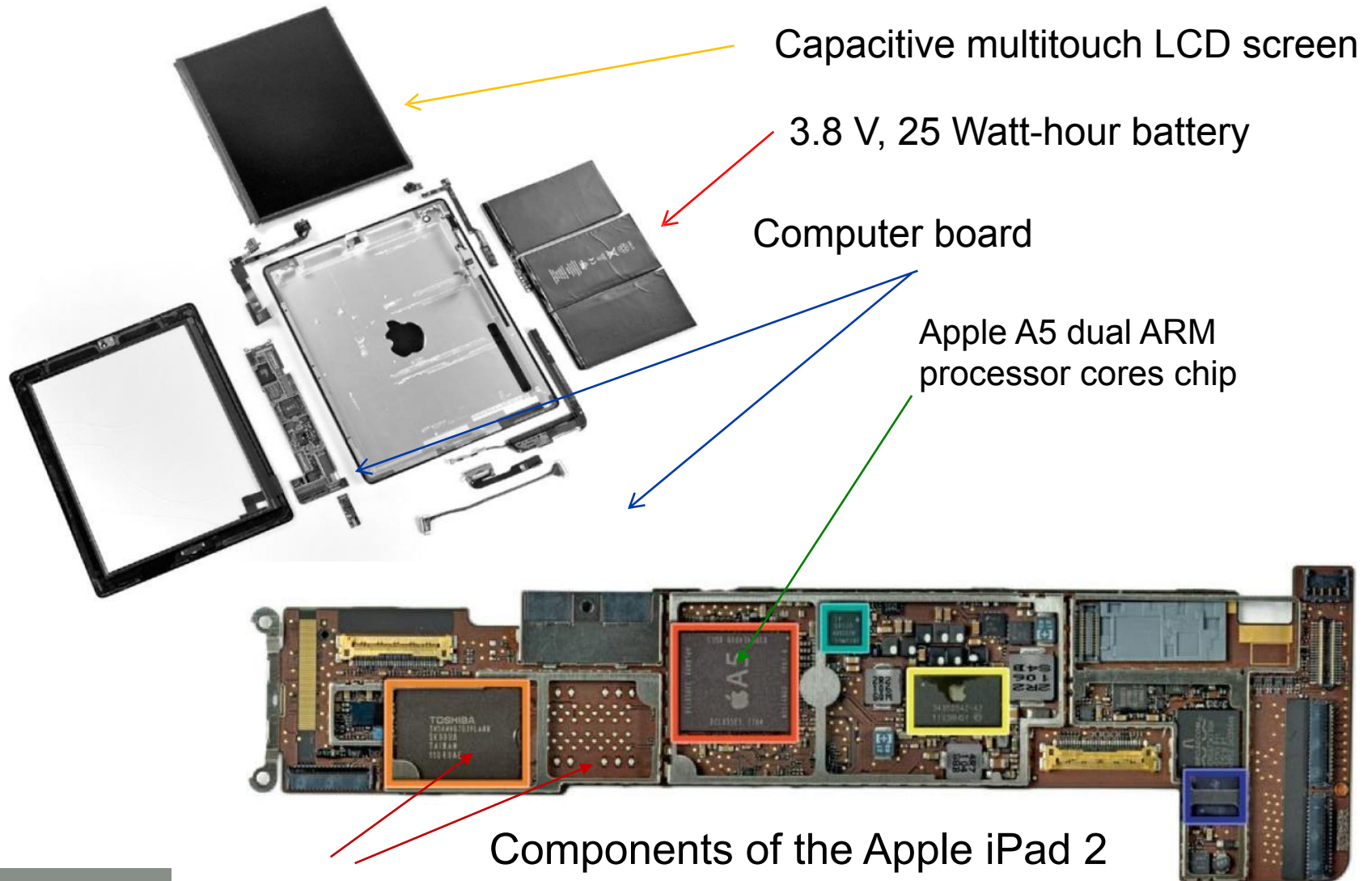


Through the Looking Glass

- LCD screen: picture elements (pixels)
- Each coordinate in the frame buffer on the left determines the shade of the corresponding coordinate for the raster scan CRT display on the right.
- Pixel (X_0, Y_0) contains the bit pattern 0011, which is a lighter shade on the screen than the bit pattern 1101 in pixel (X_1, Y_1) .

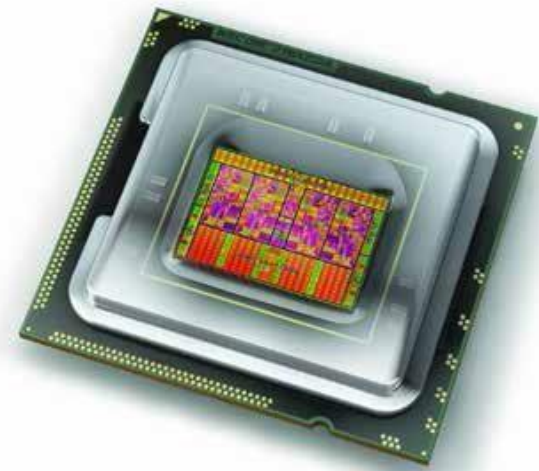
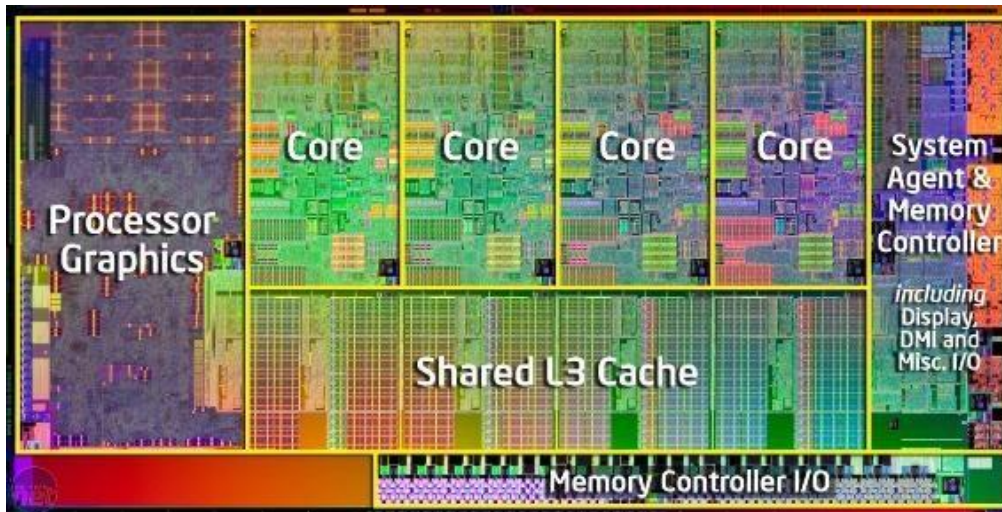


Opening the Box



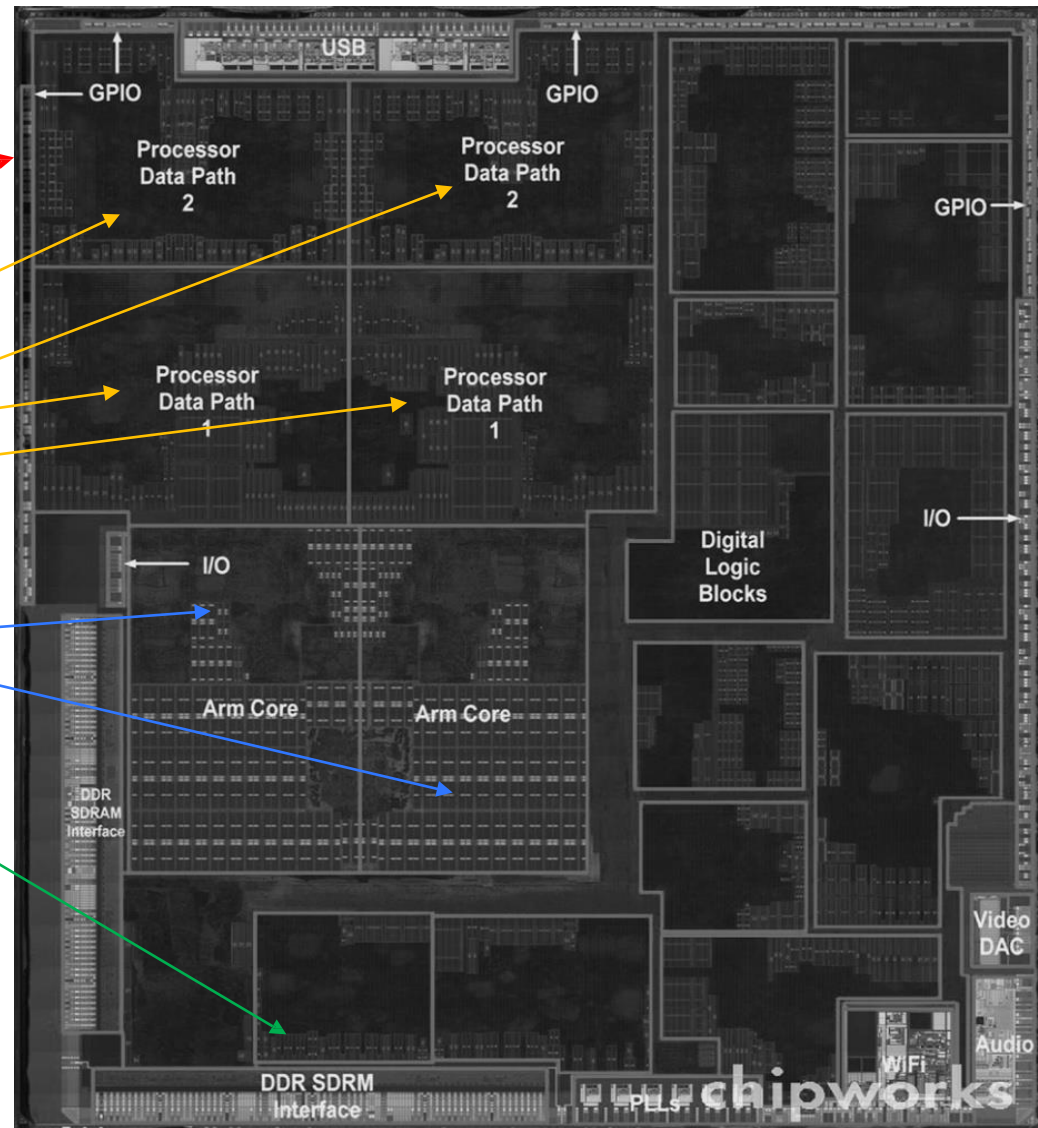
Inside the Processor (CPU)

- Datapath: performs operations on data
- Control: sequences datapath, memory, ...
- Cache memory
 - Small fast SRAM memory for immediate access to data



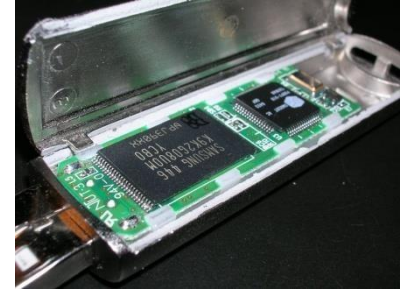
Inside the Processor (CPU)-2

- The processor IC inside Apple A5 package
 - Size of chip is 12.1 by 10.1 mm
 - Graphical processor unit (GPU) with four datapaths
 - Two identical ARM processors
 - Interfaces to main memory (DRAM)

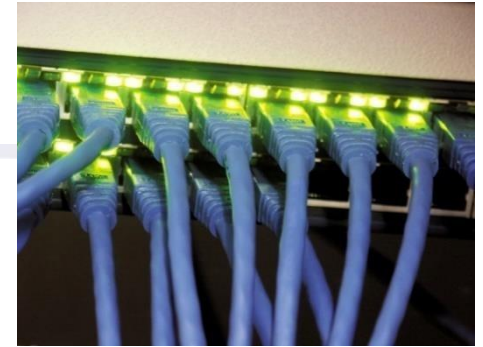


A Safe Place for Data

- Volatile main memory
 - Loses instructions and data when power off
- Non-volatile secondary memory
 - Flash memory
 - Optical disk (CDROM, DVD)
 - Magnetic disk



Networks



Backbone of computer systems

- Communication
 - Information is exchanged between computers at high speeds
- Resource sharing
 - Rather than each computer having its own I/O devices, computers on the network can share I/O devices
- Nonlocal access
 - By connecting computers over long distances, users need not be near the computer they are using

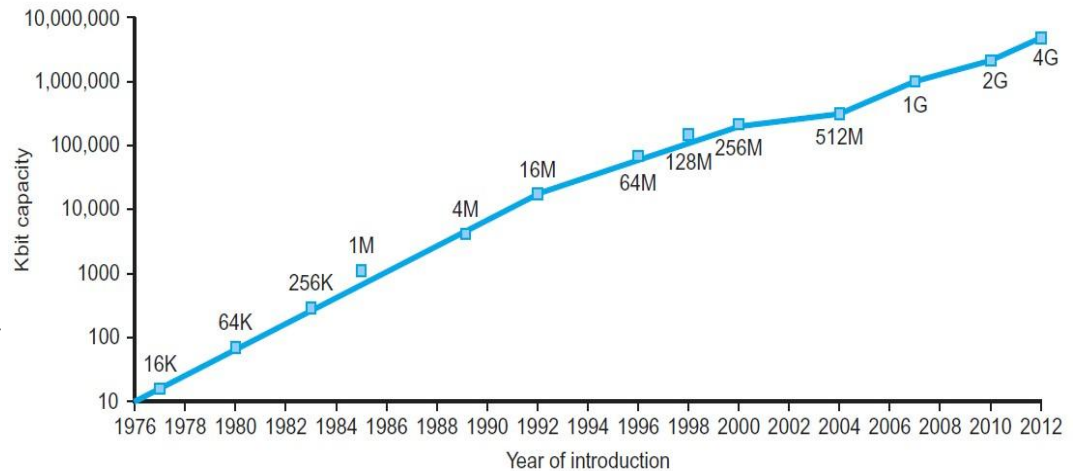
Networks (2)



- Local area network (LAN): Ethernet
 - A network designed to carry data within a geographically confined area, typically within a single building – 40 gigabits/s
- Wide area network (WAN): the Internet
 - A network extended over hundreds of kilometers that can span a continent
- Wireless network: WiFi, Bluetooth
 - transmission rates from 1 to 100 million bits per second

Technology Trends

- Electronics technology continues to evolve
 - Increased capacity and performance
 - Reduced cost

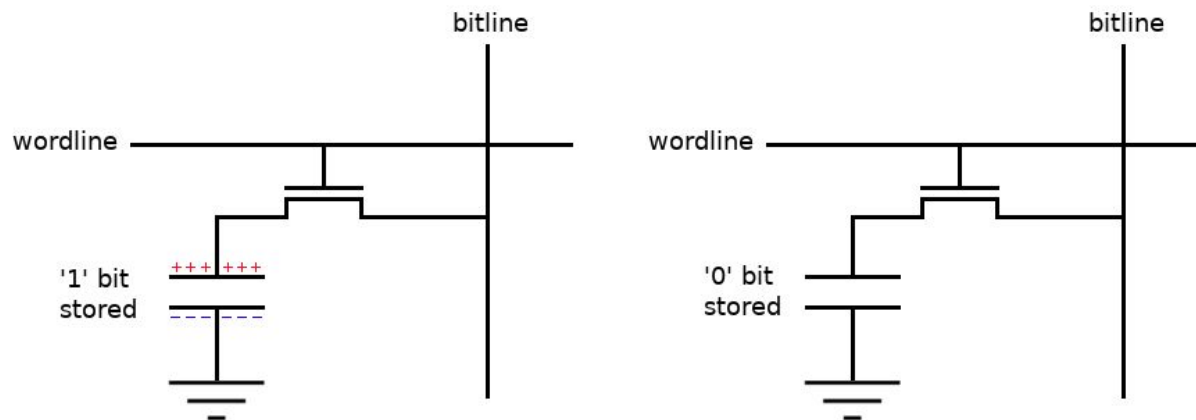


DRAM Capacity per chip over time

Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2013	Ultra large scale IC	250,000,000

Let us recap some basics- Electronics

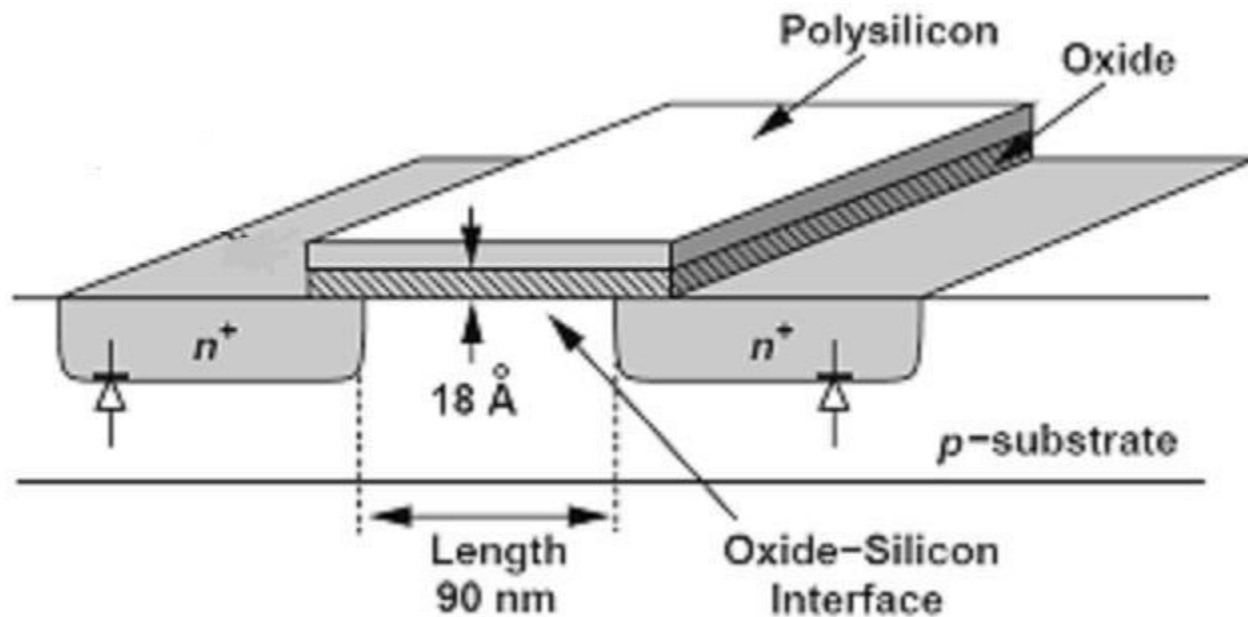
- Analog circuits (R, L, C) and Digital Circuits (Transistor)
- Flow of electrons is Current
- HI or “1” logic is 5 volt that means large amount of electrons will flow from measuring point to a common point
- LOW or “0” logic is 0 volt that means no amount of electron will flow from measuring point to a common point
- “1” logic stores enough electrons through capacitance effect using semiconducting technology that will create 5 volt potential



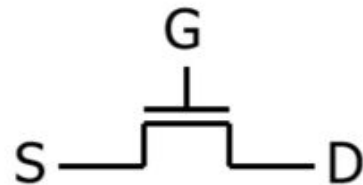
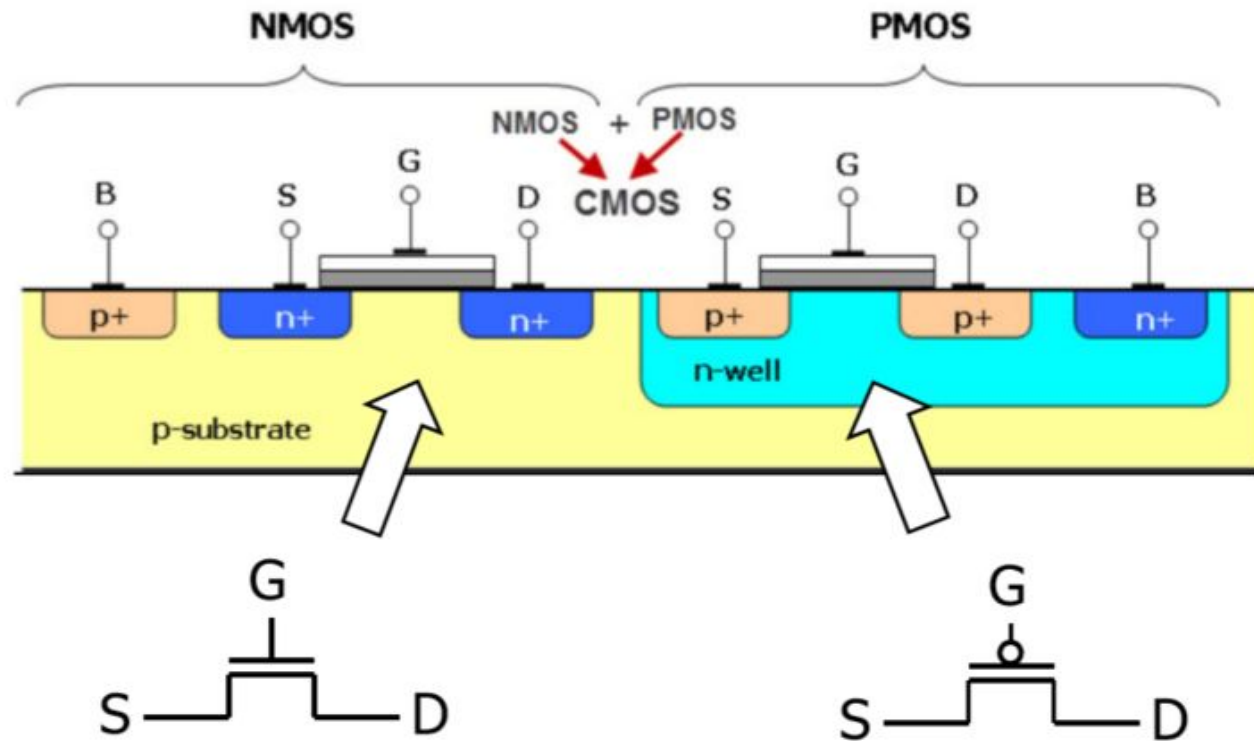
Semiconductor Technology

- Silicon: Semiconductors
- Add materials to transform properties:
 - Conductors
 - adding copper or aluminum wire
 - Insulators
 - like plastic or glass
 - Capacitors
 - adding insulators between conductors, can store electrons
 - Switches (transistors)
 - conduct or insulate under special conditions

MOSFET: Channel Length (process technology node- 90nm)

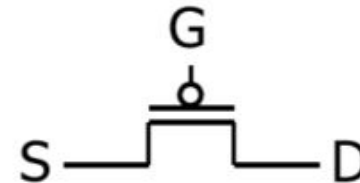


MOSFET: nMOS & pMOS



n-type

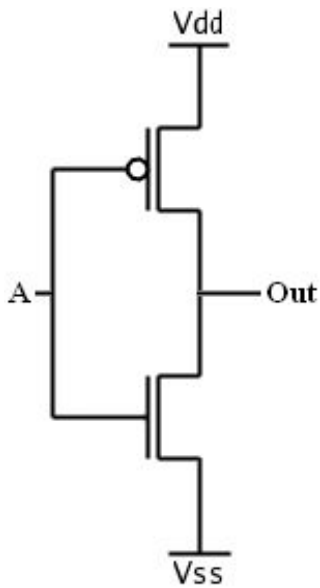
Open when voltage at G is low
Closed when voltage at G is high



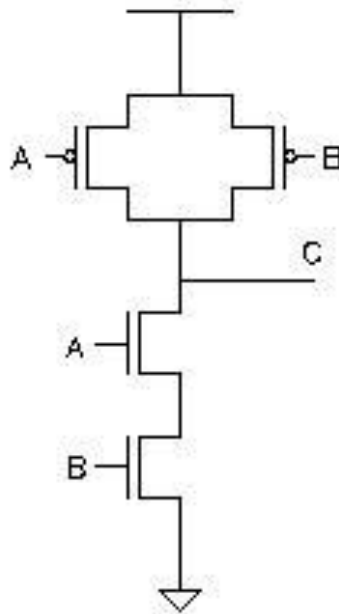
p-type

Closed when voltage at G is low
Open when voltage at G is high

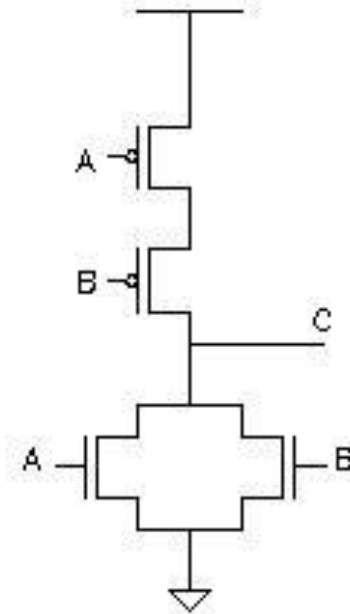
CMOS (universal): not, nand, nor



NOT



NAND



NOR

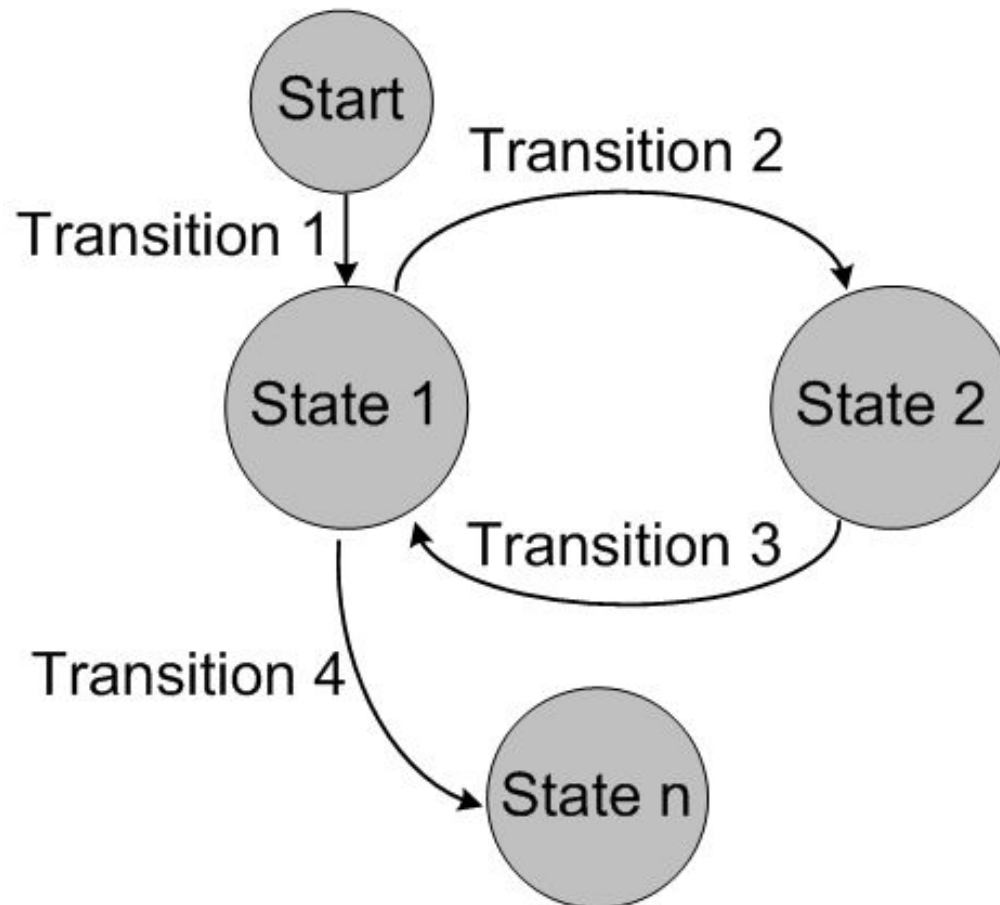
Digital Circuit Classification

- - Output depends only solely on the current combination of circuit inputs
 - Same set of input will always produce the same outputs
 - Consists of basic gates such as AND, OR, NOR, NAND, and NOT gates
-

Sequential Circuits

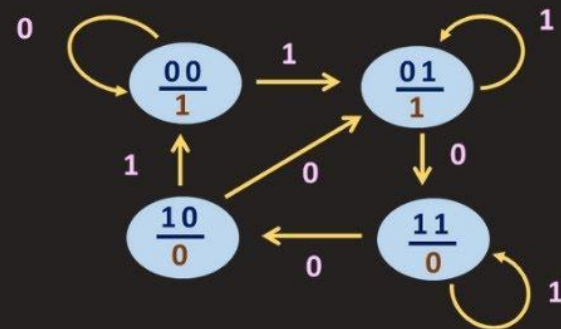
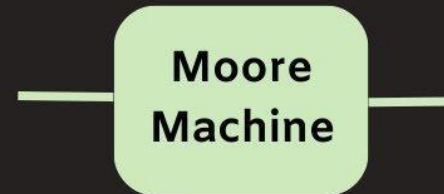
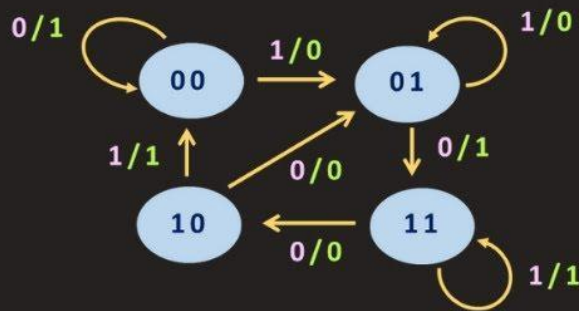
- - Output depends on the current inputs and state of the circuit (or past sequence of inputs)
 - Memory elements such as flip-flops and registers are required to store the “state”
 - Same set of input can produce completely different outputs

Sequential Circuit: State Machines

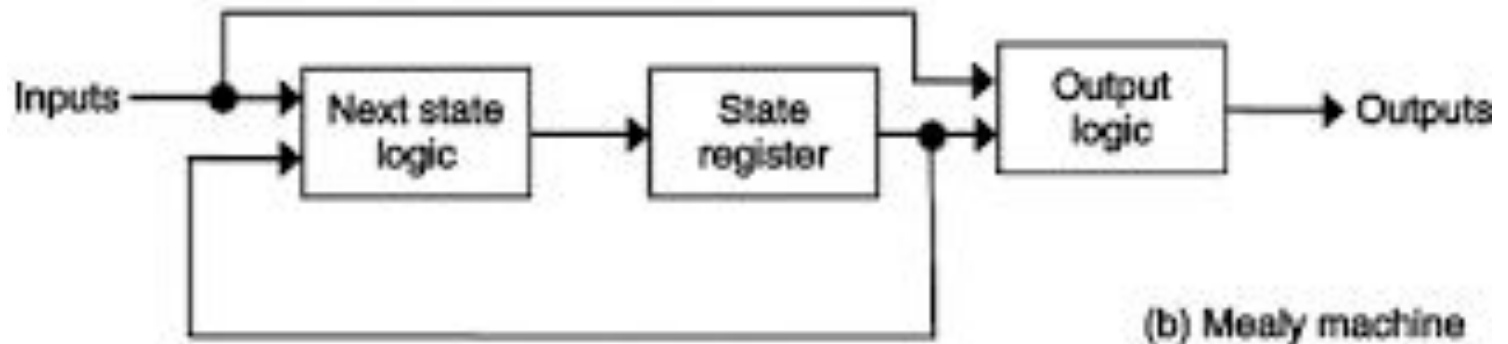
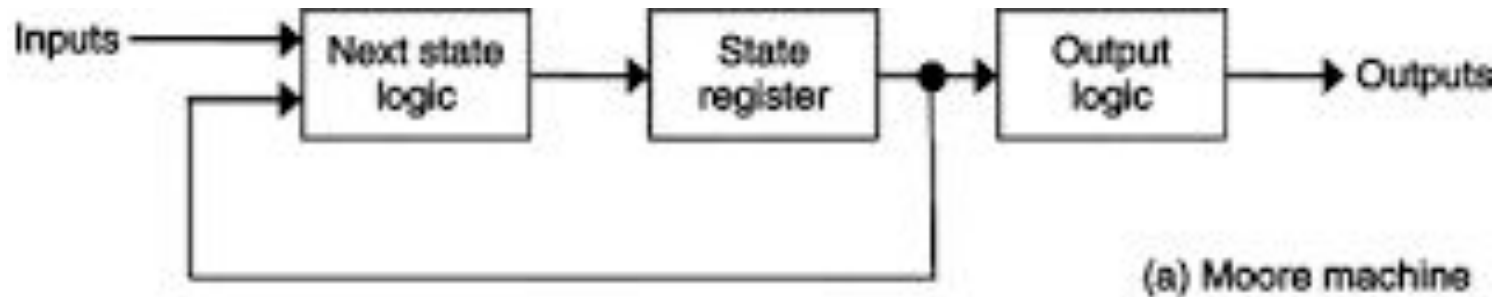


Mealy and Moore Machine

Finite State Machine

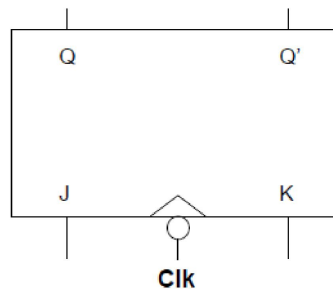


Mealy vs Moore Machine



Flipflops - Unit of Memory

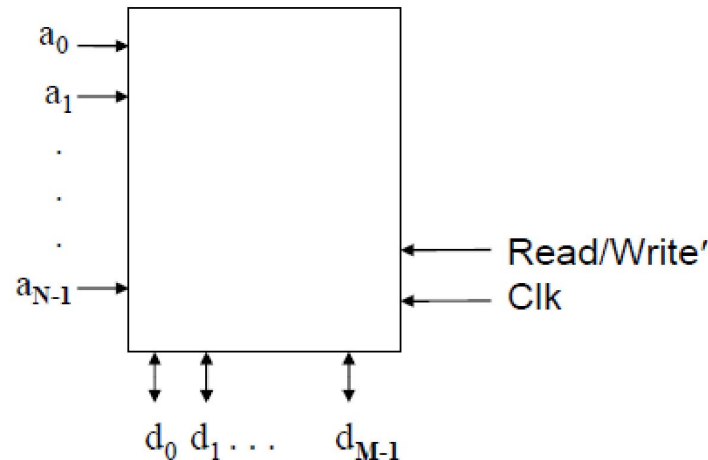
- A key memory device in microprocessors is the flip-flop.
- Remember: Flip-flops are clocked. Latches are not.
- Computers use clocked circuits => flip-flops.
- For example, a J-K flip-flop is shown as:



- In microprocessors, flip-flops are typically grouped together with common control signals into registers- the fastest and closest memory to CPU

Memory

- Another common subsystem in microprocessors is MEMORY, with multiple address lines and multiple data lines:



- The above has N address lines and M data lines. The capacity of this memory system is ?

Memory Types: ROM and RAM

Random Access Memory

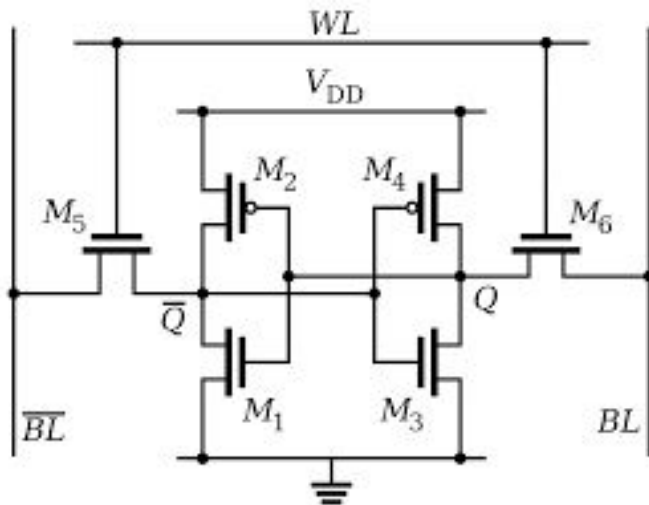
- **Dynamic Random Access Memory (DRAM):** periodic refresh is required to maintain the contents of a DRAM chip
- **Static Random Access Memory (SRAM):** no periodic refresh is required. Always more predictable and usually faster than DRAM.

Read-Only Memory

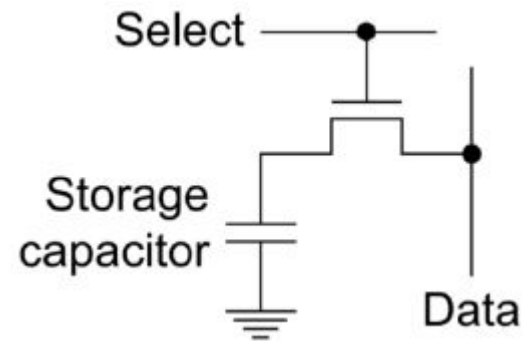
Mask-programmed read-only memory (MROM): programmed when being manufactured

Programmable read-only memory (PROM): the memory chip can be programmed by the end user

Memory: SRAM and DRAM

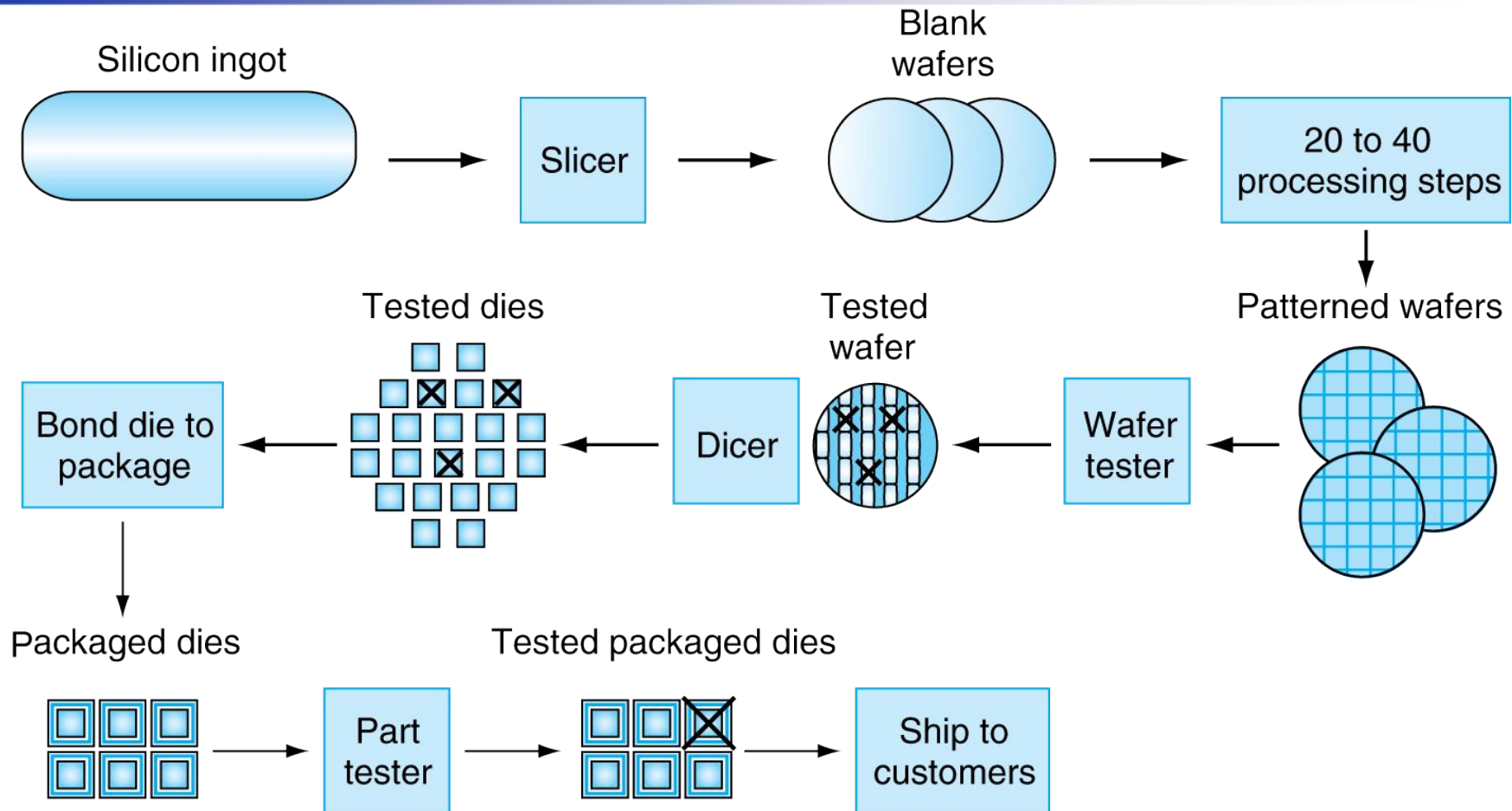


SRAM single cell



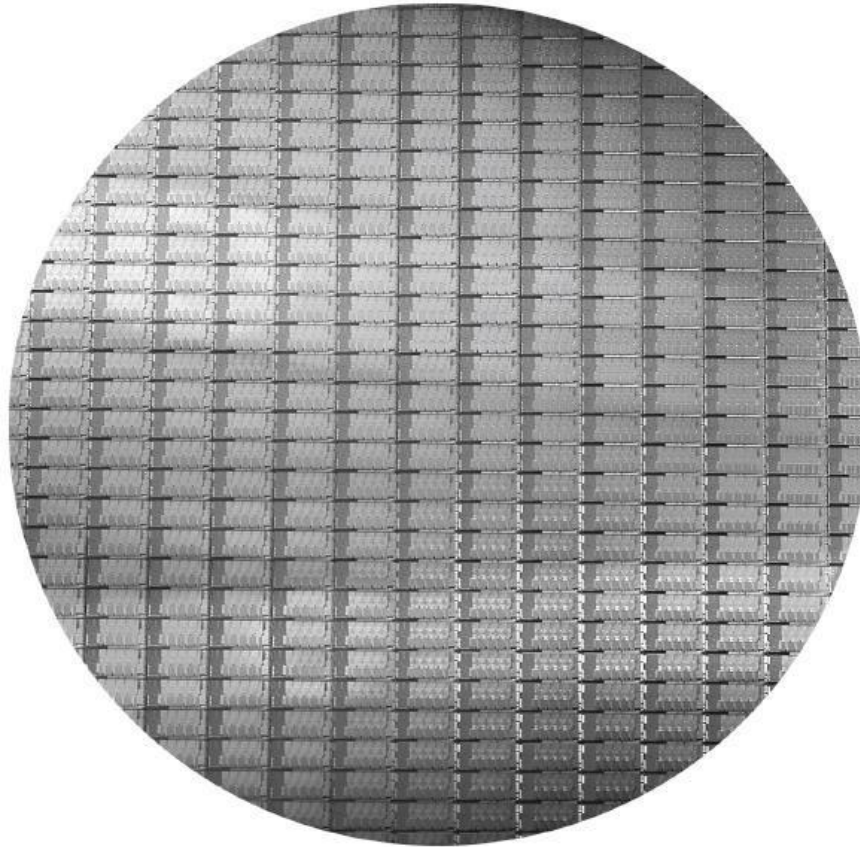
DRAM single cell

Manufacturing ICs



- **Yield**: proportion of working dies per wafer

Intel Core i7 Wafer



- 300mm wafer, 280 chips, 32nm technology
- Each chip is 20.7 by 10.5 mm

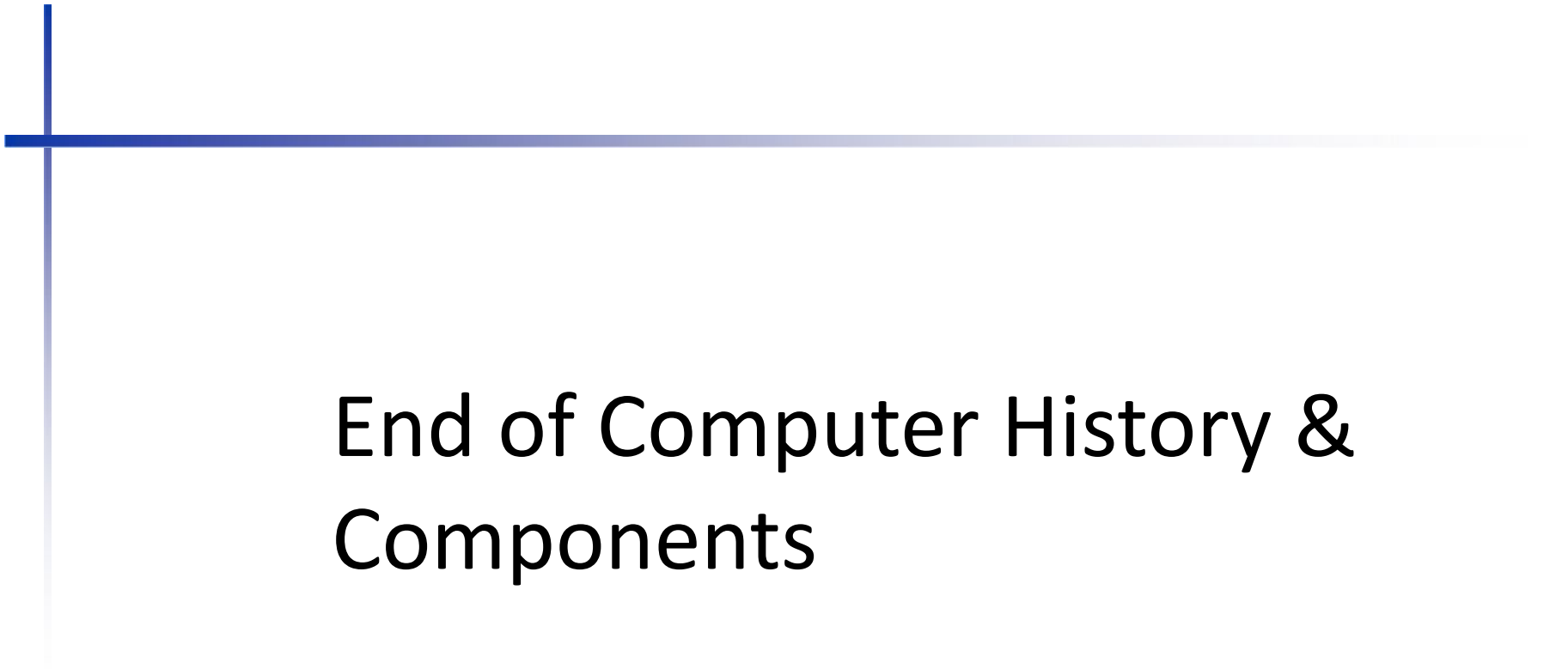
Integrated Circuit Cost

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

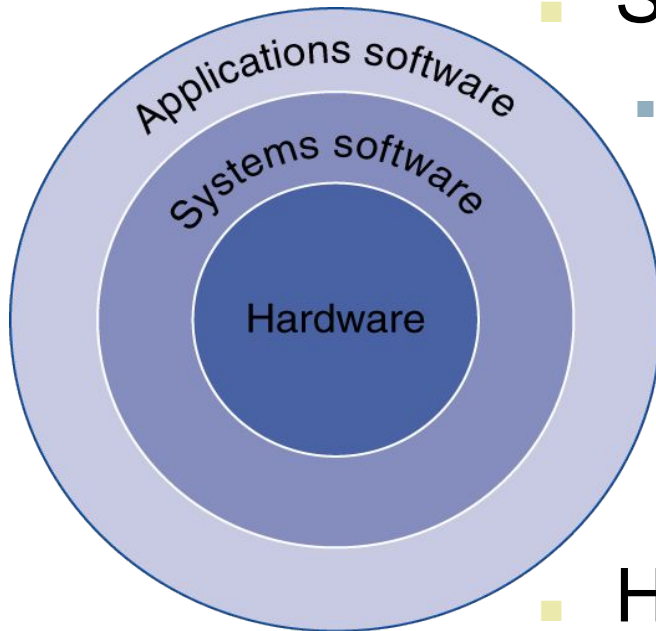
$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area}/2))^2}$$

- Nonlinear relation to area and defect rate
 - Wafer cost and area are fixed
 - Defect rate determined by manufacturing process
 - Die area determined by architecture and circuit design



End of Computer History & Components

Below Your Program



- Application software
 - Written in high-level language
- System software
 - Compiler: translates HLL code to machine code
- Operating System:
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources
- Hardware
 - Processor, memory, I/O controllers

Levels of Program Code

- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- Assembly language
 - Textual representation of instructions
- Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

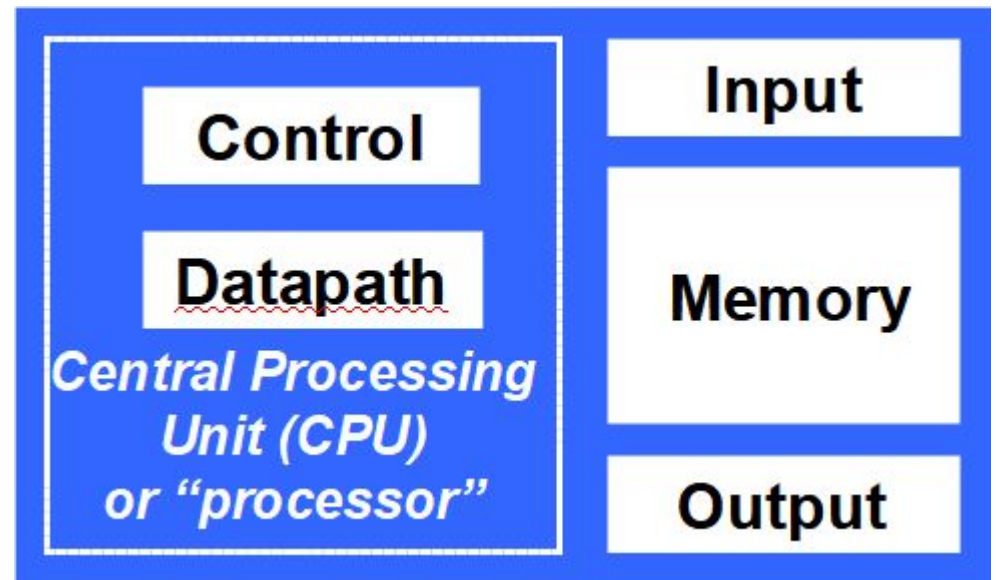
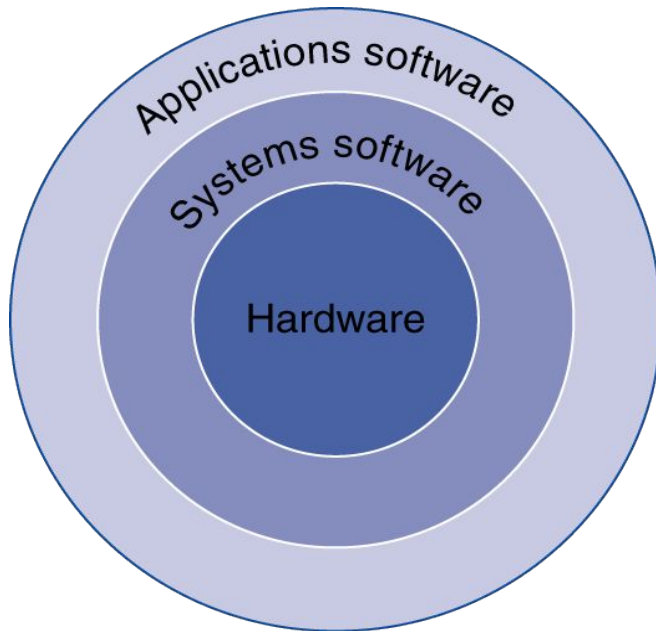
```
swap:
  muli $2, $5, 4
  add  $2, $4, $2
  lw   $15, 0($2)
  lw   $16, 4($2)
  sw   $16, 0($2)
  sw   $15, 4($2)
  jr   $31
```

Assembler

Binary machine
language
program
(for MIPS)

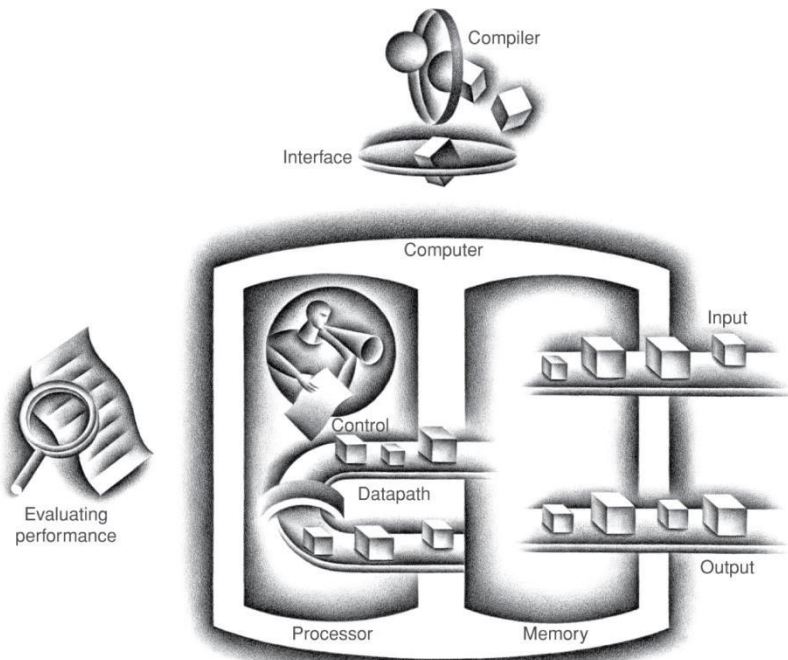
```
00000000101000010000000000011000
00000000000110000001100000100001
10001100011000100000000000000000
100011001111001000000000000000100
10101100111100100000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```

Hardware Organization of Computer



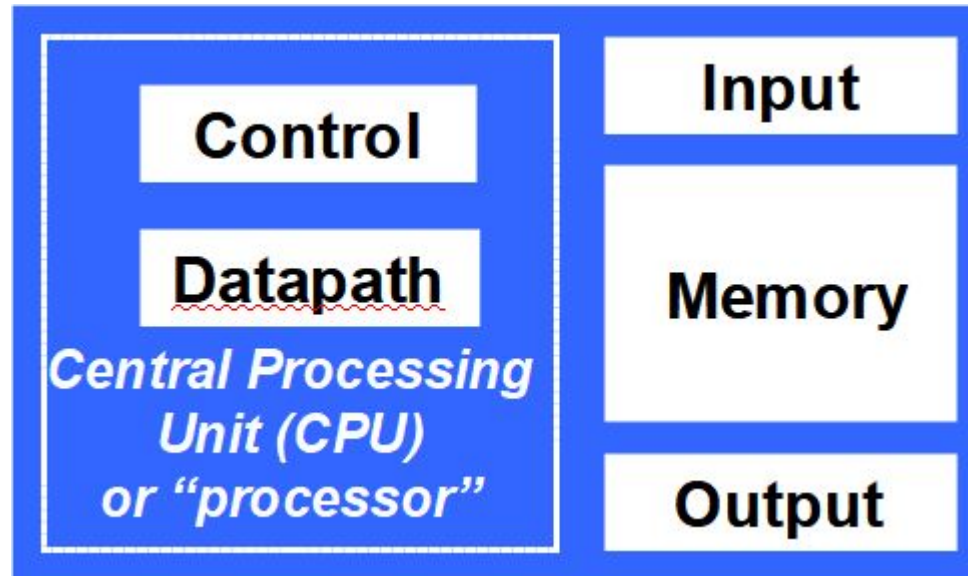
Components of a Computer

The BIG Picture



- Same components for all kinds of computer
 - Desktop, Server, Embedded
- Input/Output includes
 - User-interface devices
 - Display, keyboard, mouse
 - Storage devices
 - Hard disk, CD/DVD, flash
 - Network adapters
 - For communicating with other computers

We need a language? ISA



Instruction Set Architecture (ISA)

- A set of assembly language instructions (ISA) provides a link between software and hardware.
- Given an instruction set, software programmers and hardware engineers work more or less independently.
- ISA is designed to extract the most performance out of the available hardware technology.

Architecture and Organization

**Software
Programmers**



**Hardware
Engineers**

ISA

- Defines registers
- Defines data transfer modes between registers, memory and I/O
- Types of ISA: RISC, CISC, VLIW, Superscalar
- Examples:
 - IBM370/X86/Pentium/K6 (CISC)
 - PowerPC (Superscalar)
 - Alpha (Superscalar)
 - ARM (RISC)
 - MIPS (RISC and Superscalar)
 - Sparc (RISC), UltraSparc (Superscalar)
 - RISC V (Open Source, RISC)

Computer *Architecture*

- Architecture: System attributes that have a direct impact on the logical execution of a program
- Architecture that is visible to a programmer:
 - Instruction set
 - Data representation
 - I/O mechanisms
 - Memory addressing

Computer *Organization*

- Organization: Physical details that are transparent to a programmer, such as
 - Hardware implementation of an instruction
 - Control signals
 - Memory technology used
- Example: x86 architecture has been used by both Intel and AMD computers, which may differ in their organization.

CPU Design Project

- Design and implementation of a processor:
 - Define instruction set
 - Design datapath and control hardware
 - Implement hardware in FPGA
 - Verify

Abstractions in Comp Org & Arch

The BIG Picture

- Abstraction helps us deal with complexity
 - Hides lower-level details
- Instruction Set Architecture (ISA) or Computer Architecture
 - The hardware/software interface
 - Includes instructions, registers, memory access, I/O, and so on
- Operating system hides details of doing I/O, allocating memory from programmers

Research and Developments of Continuing Interest

- Instruction level parallelism (ILP)
- Multi-core systems and chip multi-processing (CMP)
 - Processors
 - Inter-processor communication
 - Memory organization
 - Operating system
 - Programming languages
 - Computing algorithms
- Energy efficiency and low power design
- Embedded systems
- Quantum computing, biological computing, . . .

What You Will Learn: Comp Org & Arch

- How programs are translated into machine language
 - and how hardware executes them
- The hardware/software interface
- What determines program performance
 - and how it can be improved
- How hardware designers design computer & improve performance
- What is parallel processing

Understanding Performance

- Algorithm
 - determines number of operations executed
- Programming language, compiler, architecture
 - determine number of machine instructions executed per operation
- Processor and memory system
 - determine how fast instructions are executed
- I/O system (including OS)
 - determine how fast I/O operations are executed

Eight Great Design Ideas

■ Design for ***Moore's Law***



- Computer designs can take years, resources available per chip can easily double or quadruple between start and finish of project
- Anticipate where technology will be when design finishes

■ Use ***Abstraction*** to simplify design



- Hide lower-level details to offer a simpler model at higher levels

■ Make the ***Common Case Fast***



- To enhance performance better than optimizing the rare case

Eight Great Design Ideas (2)

■ Performance *via Parallelism*



- A form of computation in which many calculations are carried out simultaneously

■ Performance *via Pipelining*



- A particular pattern of parallelism
- A set of data processing elements connected in series, so that the output of one element is the input of the next one

■ Performance *via Prediction*



- It can be faster on average to guess and start working rather than wait

Eight Great Design Ideas (3)

■ ***Hierarchy*** of memories



- The closer to the top, the faster and more expensive per bit of memory
- The wider the base of the layer, the bigger the memory

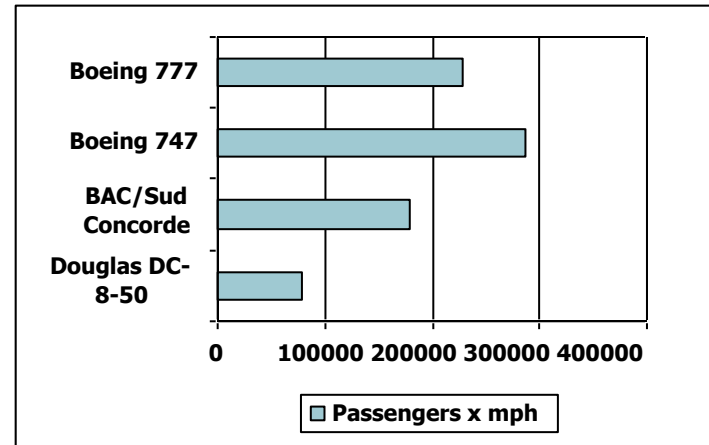
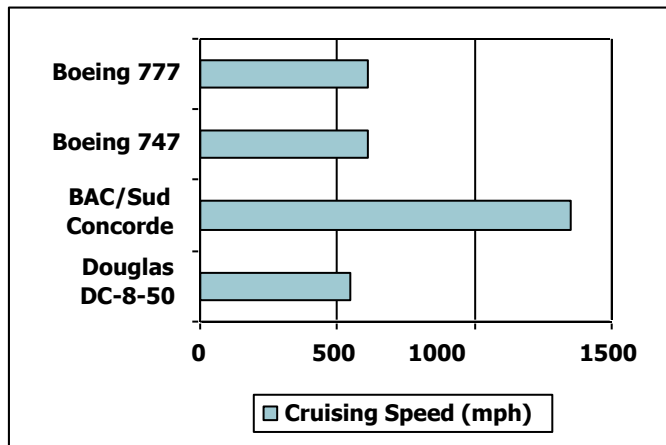
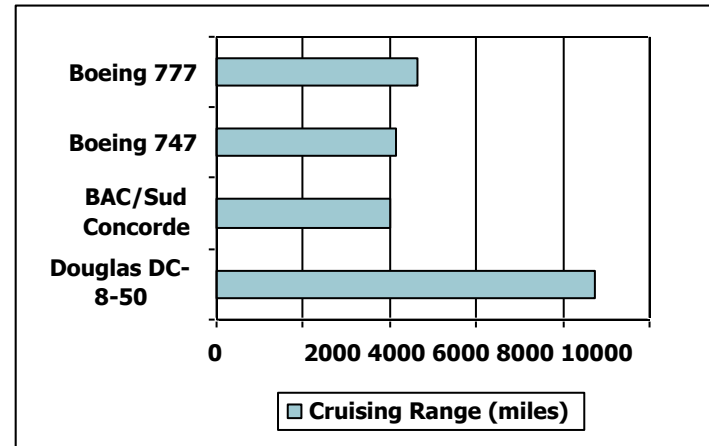
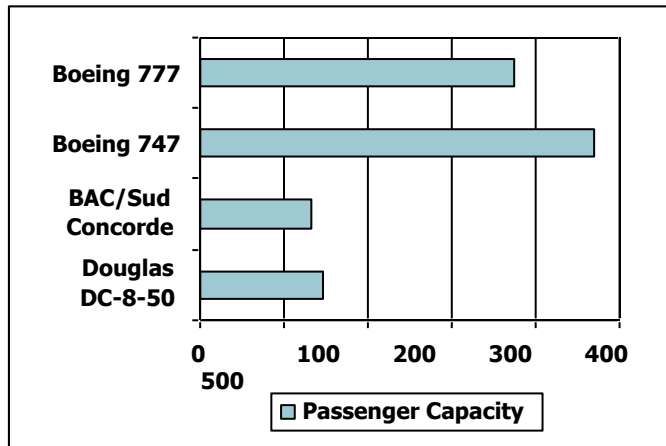
■ ***Dependability*** via redundancy



- Design systems dependable by including redundant components
 - to take over when failure occurs, and
 - to help detect failures

Defining Performance

- Which airplane has the best performance?



Response Time and Throughput

- Response time
 - How long it takes to do a task
- Throughput
 - Total work done per unit time
e.g., tasks/transactions/... per hour
- How are response time and throughput affected by
 - Replacing the processor with a faster version?
 - Adding more processors?
- We'll focus on response time for now...

Relative Performance

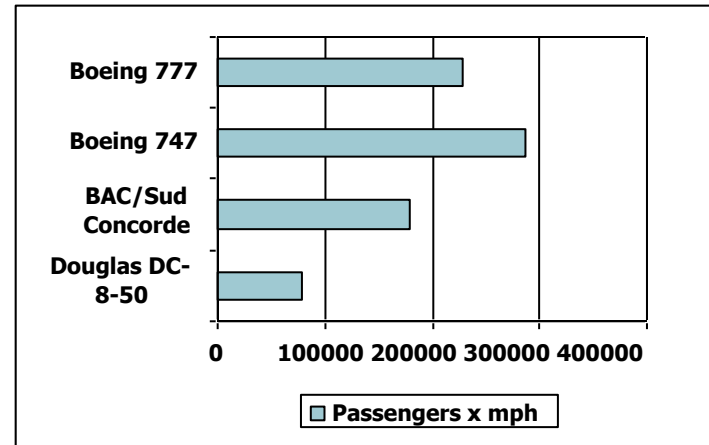
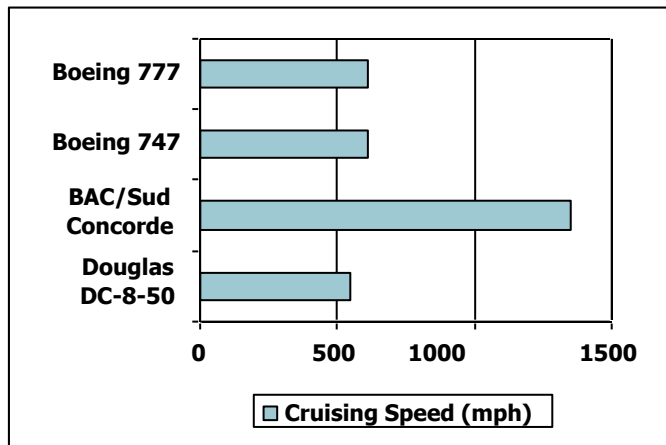
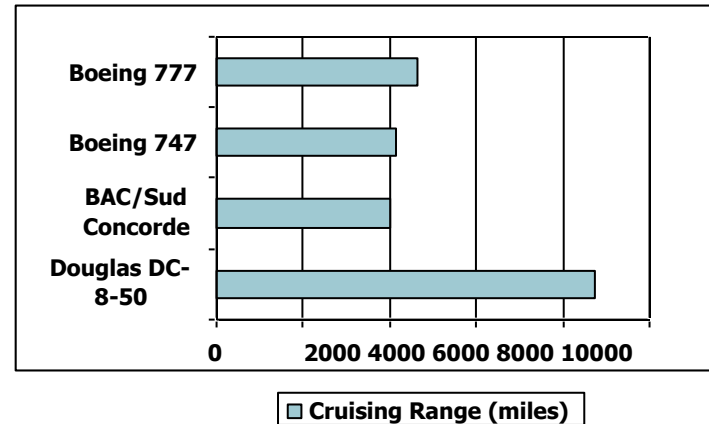
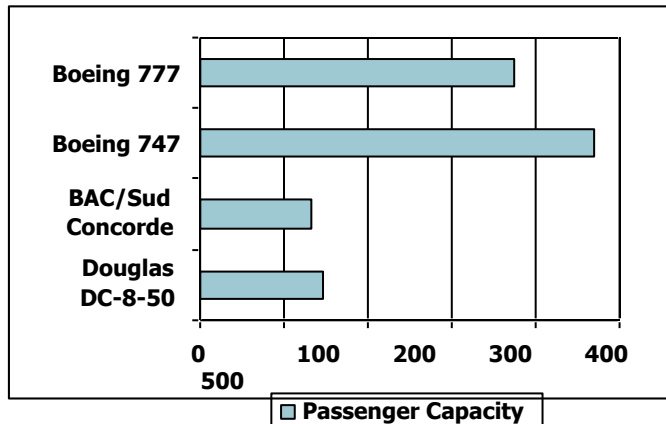
- Define Performance = $1/\text{Execution Time}$
- “X is n times faster than Y”

$$\begin{aligned} \text{Performance}_X / \text{Performance}_Y \\ = \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

- Example: time taken to run a program
 - 10s on A, 15s on B
 - $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15\text{s} / 10\text{s} = 1.5 = 1\frac{1}{2}$
 - So A is $1\frac{1}{2}$ times faster than B

Defining Performance

- Which airplane has the best performance?



Response Time and Throughput

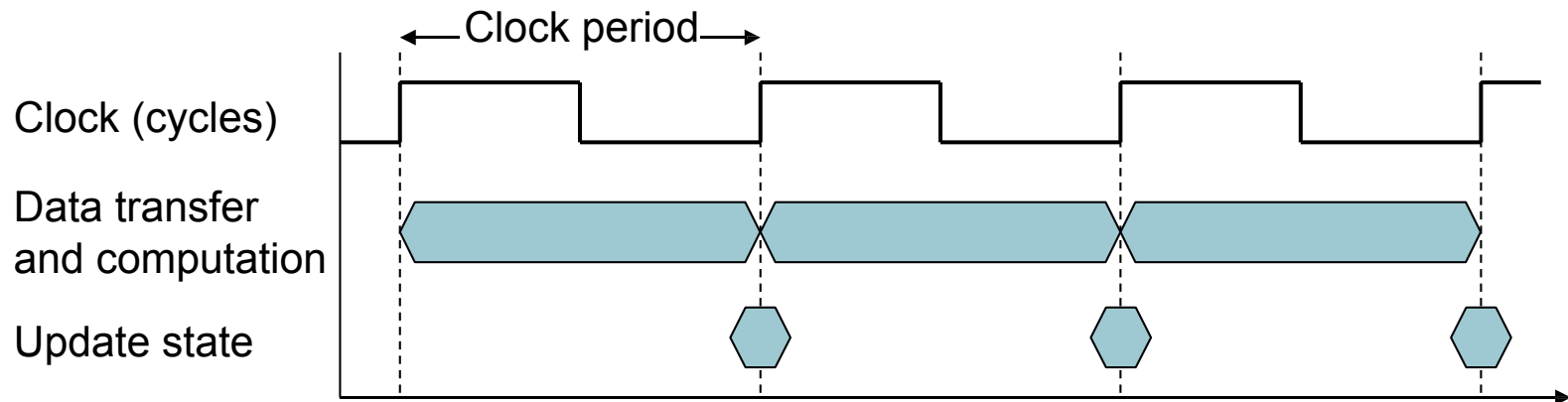
- Response time
 - How long it takes to do a task
- Throughput
 - Total work done per unit time
e.g., tasks/transactions/... per hour
- How are response time and throughput affected by
 - Replacing the processor with a faster version?
 - Adding more processors?
- We'll focus on response time for now...

Measuring Execution Time

- Elapsed time
 - Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
 - Determines system performance
- CPU time
 - Time spent processing a given job
 - Minus I/O time, other jobs' shares
 - Includes user CPU time and system CPU time
 - Different programs are affected differently by CPU and system performance
 - Running on servers – I/O performance – hardware and software
 - Total elapsed time is of interest
 - Define performance metric and then proceed

CPU Clocking

- Operation of digital hardware governed by a constant-rate clock



- Clock period: duration of a clock cycle
 - e.g., $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$
- Clock frequency (rate): cycles per second
 - e.g., $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$

CPU Time

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

- Performance can be improved by
 - Reducing number of clock cycles
 - Increasing clock rate
 - Hardware designer must often trade off clock rate against cycle count

CPU Time Example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
 - Aim for 6s CPU time
 - Can do faster clock, but causes $1.2 \times$ clock cycles
- How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$

Instruction Count and CPI

$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles Per Instruction}$

$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction Count for a program
 - Determined by program, ISA, and compiler
- Average cycles per instruction
 - Determined by CPU hardware
 - If different instructions have different CPI
 - Average CPI gets affected by instruction mix (dynamic frequency of instructions)

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster? by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps}\end{aligned}$$

A is faster...

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}\end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2$$

...by this much

CPI in More Detail

- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

- Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \underbrace{\frac{\text{Instruction Count}_i}{\text{Instruction Count}}}_{\text{Relative frequency}} \right)$$

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5
 - Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
 - Avg. CPI = $10/5 = 2.0$
- Sequence 2: IC = 6
 - Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$
 - Avg. CPI = $9/6 = 1.5$

Performance Summary

The BIG Picture

CPU Time = Seconds/Program

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock Cycle}}$$

■

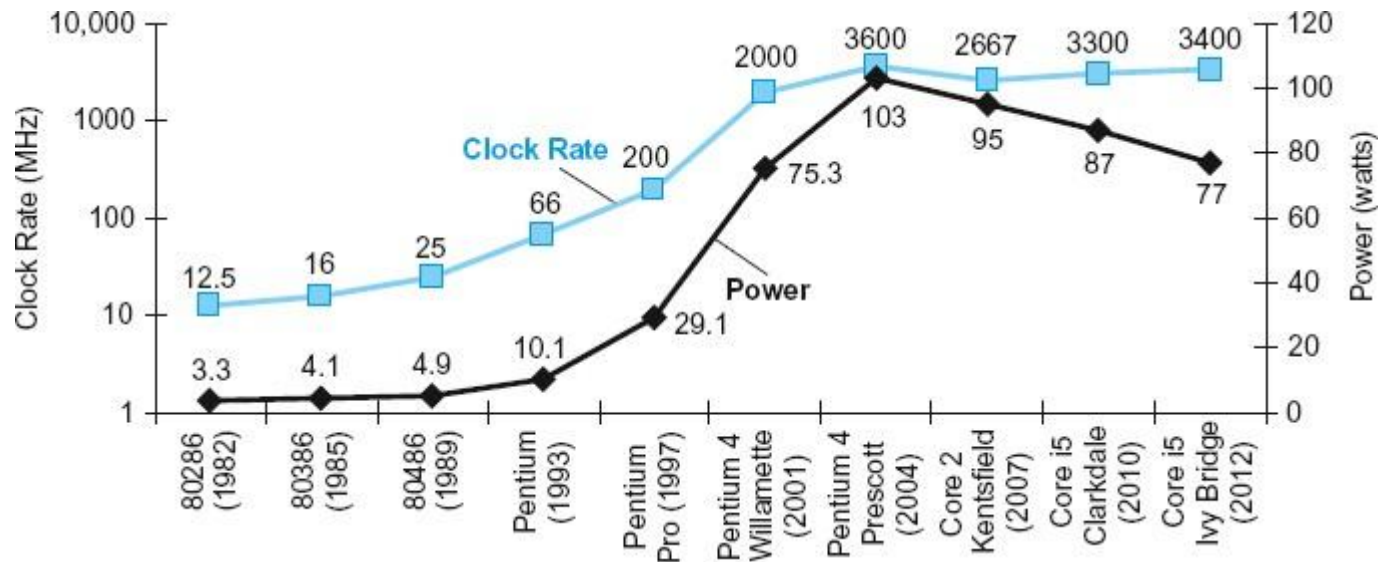
Components of performance	Units of measure
CPU execution time for a program	Seconds for the program
Instruction count	Instructions executed for the program
Clock cycles per instruction (CPI)	Average number of clock cycles per instruction
Clock cycle time	Seconds per clock cycle

FIGURE 1.15 The basic components of performance and how each is measured.

Performance Summary

- Performance depends on
 - Algorithm: affects IC, possibly CPI
 - Programming language: affects IC, CPI
 - Compiler: affects IC, CPI
 - Instruction set architecture: affects IC, CPI, T_c

Power Trends



Clock rate and Power for Intel x86 microprocessors over eight generations

■ In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30

5V → 1V

×1000

Reducing Power

- Suppose a new CPU has
 - 85% of capacitive load of old CPU
 - 15% voltage and 15% frequency reduction

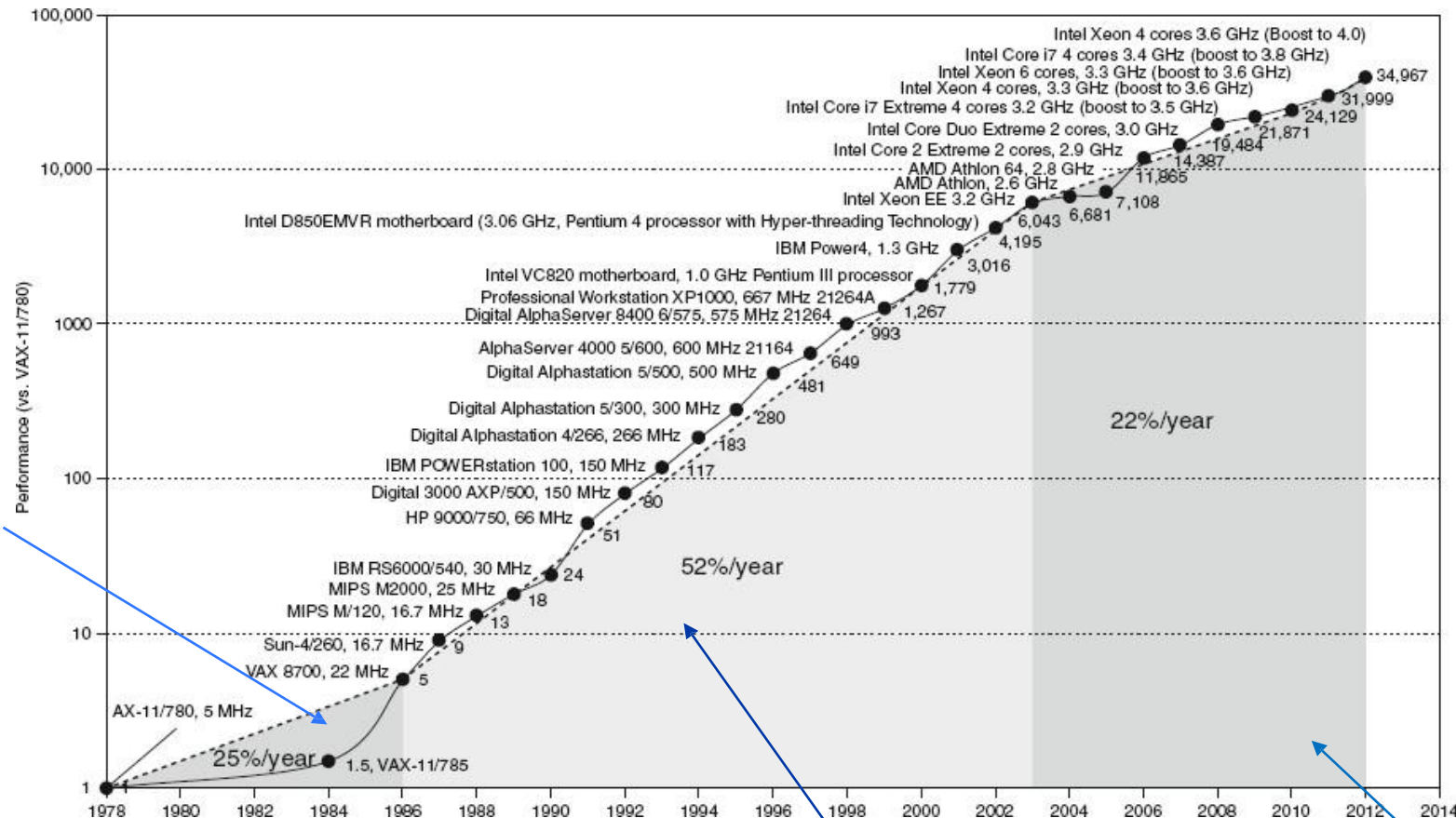
$$\frac{\text{Power}_{\text{new}}}{\text{Power}_{\text{old}}} = \frac{\langle \text{Capacitive load} \times 0.85 \rangle \times \langle \text{Voltage} \times 0.85 \rangle^2 \times \langle \text{Frequency switched} \times 0.85 \rangle}{\text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}}$$

Thus the power ratio is

$$0.85^4 = 0.52$$

- The power wall
 - We can't reduce voltage further
 - We can't remove more heat
- How else can we improve performance?

Uniprocessor Performance



Technology driven

Advanced architectural and organizational ideas

Constrained by power, instruction-level parallelism, memory latency

Multiprocessors

- Multicore microprocessors
 - More than one processor per chip
- Requires explicitly parallel programming
 - Compare with instruction level parallelism
 - Hardware executes multiple instructions at once
 - Hidden from the programmer
 - Hard to do (Why?)
 - Programming for performance
 - Load balancing
 - Optimizing communication and synchronization

Concluding Remarks

- Cost/performance is improving
 - Due to underlying technology development
- Hierarchical layers of abstraction
 - In both hardware and software
- Instruction set architecture
 - The hardware/software interface
- Execution time: the best performance measure
- Power is a limiting factor
 - Use parallelism to improve performance

Acknowledgement

The slides are adopted from Computer Organization and Design, 5th Edition

by David A. Patterson and John L. Hennessy 2014,
published by MK (Elsevier)

